



LABORATORIO DE ANÁLISIS DE SISTEMAS Y SEÑALES

SIMULADOR DE FUNCIONES DE TRANSFERENCIA DISCRETAS DE 1ERO Y 2DO ORDEN

Índice

1.	Tema de Estudio del Programa de la Asignatura	4
2.	Título de la Práctica	4
3.	Autores	4
4.	Objetivos del Aprendizaje	4
5.	Recursos	5
6.	Fundamento Teórico	6
7.	Cuestionario Previo	13
8.	Desarrollo de Actividades de la Práctica	
8.1	Simulación de un Filtro Pasa Bajas de Primer Orden	15
8.2	Simulación de un Filtro Pasa Altas de Primer Orden	17
8.3	Simulación de un Filtro Pasa Banda de Segundo Orden	18
8.4	Simulación de un Sistema de Segundo Orden Subamortiguado	19
9.	Solución de las Actividades de la Práctica	
9.1	Solución de la Actividad 8.1	24
9.2	Solución de la Actividad 8.2	25
9.3	Solución de la Actividad 8.3	26
9.4	Solución de la Actividad 8.4	27
10.	Apéndices:	
10.1	Descripción del Módulo de Evaluación basado en la Raspberry Pi Pico 2 (RP2350)	
10.2	Descripción del Programa Residente en la RP2350	
10.2	Configuración del Módulo Bluetooth HC05	
10.3	Descripción de la Interfaz Usuario de MATLAB	

1. Tema de Estudio del Programa de la Asignatura

En la asignatura de Análisis de Sistemas y Señales para las carreras de Electrónica y Aeroespacial cubre los siguientes temas:

3.2.3 Función de transferencia de sistemas de tiempo discreto.

3.3.3 Obtención de la respuesta completa de los sistemas lineales e invariantes.

En la asignatura de Análisis de Sistemas y Señales para la carrera de Computación:

3.5 Función de transferencia de sistemas en tiempo discreto

6.3 Cálculo de la respuesta en frecuencia mediante simulación digital.

2. Título de la Práctica

“Simulación de Funciones de Transferencia Discretas de Primer y Segundo Orden en Tiempo y Frecuencia”.

3. Autor

M. en I. José Antonio de Jesús Arredondo Garza

4. Objetivo del Aprendizaje

El alumno podrá manipular “*Funciones de Transferencia Discretas de Primero y Segundo Orden*” usando una tasa de muestreo fija de 8 KHz y observará su respuesta en tiempo y frecuencia utilizando una interfaz de MATLAB en donde podrá asignar coeficientes de la función, y la frecuencia de la señal de entrada.

Con respecto a la señal de entrada, esta será una onda cuadrada cuya frecuencia podrá ser manipulada, permitiéndole al alumnado ver como responde su sistema a diferentes frecuencias.

5. Recursos

Para la realización de la práctica se deberá contar con la siguiente infraestructura:

- Computadora Personal con el programa MATLAB instalado.
- Módulo de Evaluación del Microcontrolador Raspberry Pi Pico 2 (RP2350), el cual, deberá tener integrado y configurado su respectivo módulo bluetooth HC05. Así mismo, el módulo RP2350 deberá tener grabado su respectivo programa. Esta tarea deberá ser realizada previamente por el responsable del laboratorio. A continuación, se presenta un esquema del módulo RP2350 + HC05:



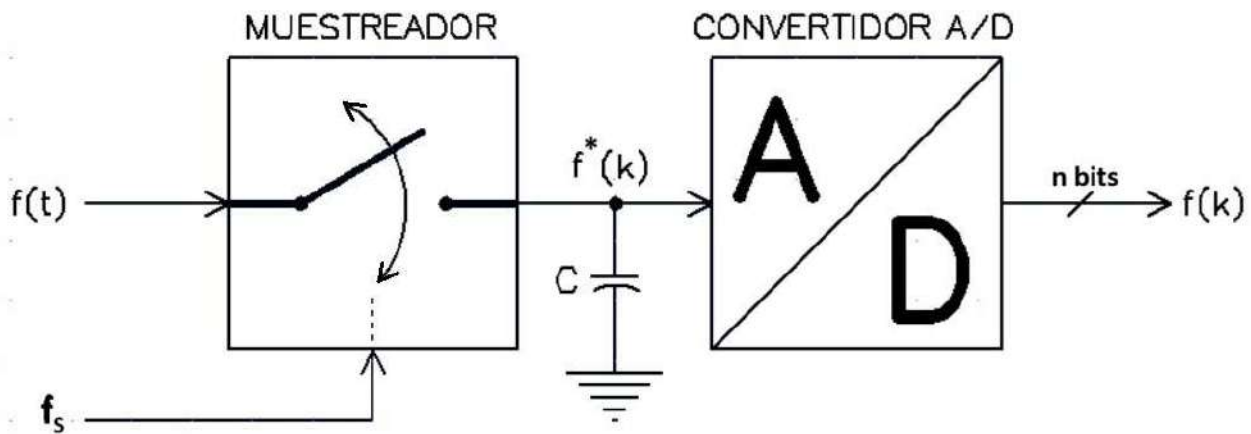
NOTA: el Pin GPIO3 (onda cuadrada), deberá estar conectado al Pin GPIO26 (canal AN0 del convertidor A/D).

- Previamente, se deberá tener configurar el módulo bluetooth a una tasa de baudaje de 460800 y con un nombre particular para que pueda ser reconocido por la computadora personal. El nombre del dispositivo se le podrá declarar en la interfaz usuario de MATLAB. Se deberá declarar su nombre en los comandos del Sistema Operativo de Windows.
- Cable USB tipo A (macho) – USB micro (macho).
- Al estar residente en la RP2350 el programa que controla la adquisición de datos via MATLAB – RP2350 y la rutina que da solución a la ecuación en diferencias, es posible tener el módulo con una alimentación de 5V a su terminal USB micro y de esta manera, no es necesario conectarlo a la PC (esto ahorraría el uso del cable USB).
- Para la justificación de experimentos en los reportes el alumno deberá hacer capturas de pantalla.

6. Fundamento Teórico

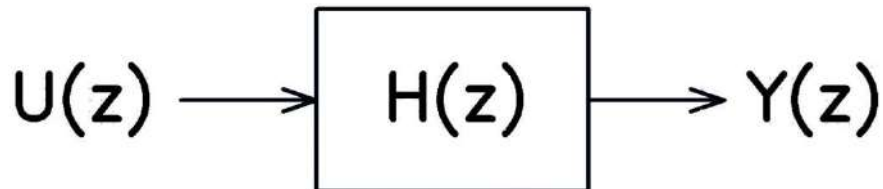
Desde la aparición de las computadoras se fueron planteando la necesidad de emular el comportamiento de sistemas y funciones provenientes del mundo analógico. Para tal fin, se necesito desarrollar nuevas herramientas matemáticas que permitieran implementar un alias equivalente que se comportara análogamente a su contraparte del mundo continuo.

Se tuvieron que desarrollar interfaces que convirtieran una variable analógica a un equivalente numérico que una computadora pueda entender y otra interfaz que inversamente pueda convertir un factor numérico a su equivalente analógico. Por esta razón, se diseñó el Convertidor Analógico – Digital (A/D) y el Convertidor Digital – Analógico (D/A). Estos dispositivos son los encargados de convertir una variable dada en voltios a un equivalente numérico y viceversa. Sin embargo, para garantizar la estabilidad de la conversión del A/D, es necesario agregar un muestreador – reten.



Es obvio que una entrada analógica $f(t)$ es convertida en una señal discreta $f(k)$ mediante el muestreador – retén en combinación con el convertidor analógico – digital y $f(z)$ mediante el uso de la Transformada Z.

Un sistema discreto (al igual que todo sistema), requiere de una señal de entrada $f(z)$ para excitar a una función de transferencia $H(z)$, dando como resultado una salida $Y(z)$. Esto es:



Genéricamente en esta práctica se estará usando una función de transferencia de segundo orden, aunque más adelante se explicará cómo usar una función de transferencia de primer orden:

$$H(z) = \frac{A0 \cdot Z^2 + A1 \cdot Z + A2}{Z^2 + B1 \cdot Z + B2} = \frac{Y(z)}{U(z)} \dots\dots\dots 1$$

Para obtener la ecuación en diferencias necesitamos dividir entre Z^2 , con lo cual obtenemos:

$$H(z) = \frac{A_0 Z^0 + A_1 Z^{-1} + A_2 Z^{-2}}{Z^0 + B_1 Z^{-1} + B_2 Z^{-2}} = \frac{Y(z)}{U(z)} \quad \text{..... 2}$$

Descomponiendo la función de transferencia en términos de entradas y salidas se tiene:

$$Y(z)[Z^0 + B_1 Z^{-1} + B_2 Z^{-2}] = U(z)[A_0 Z^0 + A_1 Z^{-1} + A_2 Z^{-2}] \quad \text{..... 3}$$

Para obtener las anti transformadas correspondientes hay que tomar en cuenta:

$f(z) * Z^0$	$\leftrightarrow$	$f(k)$	Evento actual
$f(z) * Z^{-n}$	$\leftrightarrow$	$f(k - 1)$	Eventos pasados
$f(z) * Z^{+n}$	$\leftrightarrow$	$f(k + 1)$	Eventos futuros

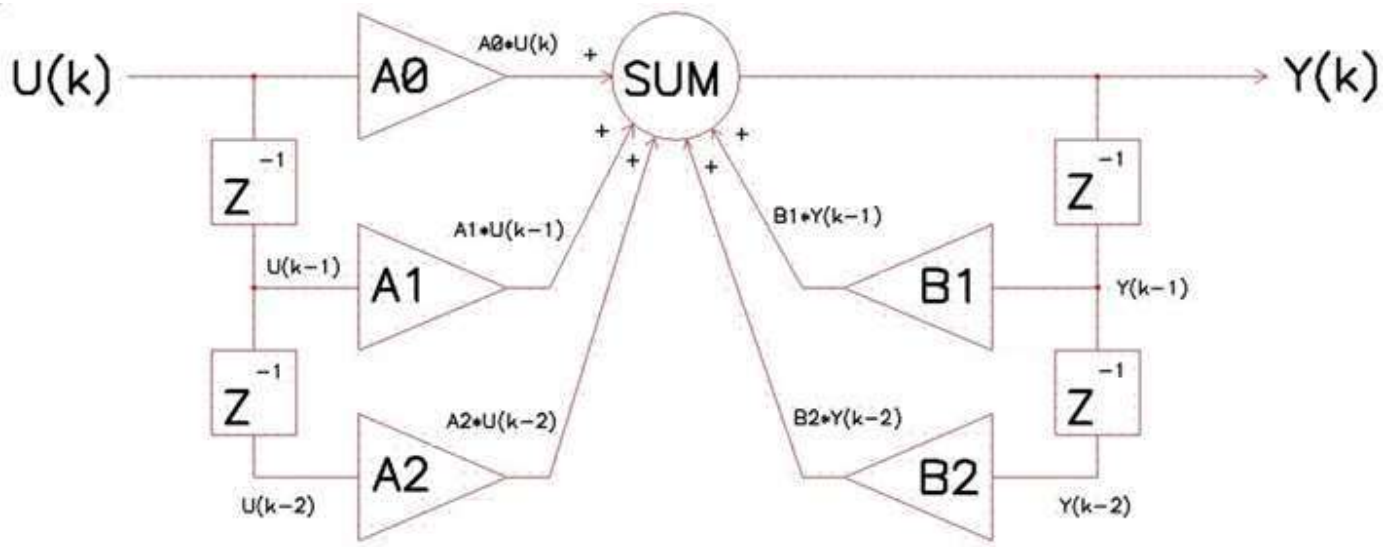
En base a la mención anterior se obtiene entonces:

$$Y(k) + B_1 * Y(k - 1) + B_2 * Y(k - 2) = A_0 * U(k) + A_1 * U(k - 1) + A_2 * U(k - 2) \quad \text{..... 4}$$

Despejando la salida actual se obtiene entonces la ecuación en diferencias:

$$Y(k) = A_0 * U(k) + A_1 * U(k - 1) + A_2 * U(k - 2) - B_1 * Y(k - 1) - B_2 * Y(k - 2) \quad \text{..... 5}$$

El diagrama de flujo correspondiente:



Aproximadamente en el año de 1976, muy previo a la aparición de los Procesadores Digitales de Señales “**DSP**” (año de 1982), la compañía “*Advance Micro Devices*” lanzó al mercado unas compuertas muy especializadas para implementar este tipo de esquemas, consistentes en “*multiplicador paralelo – serie*”, “*sumadores seriales*” y para crear los retardos Z^{-1} se emplearon “*flip – flops tipo D*”. Toda esta tarea se realizaba en forma de datos serie (*strings*) y utilizaron notación aritmética en punto fijo. Es muy importante hacer notar que la aritmética inicial de los DSP’s desarrollados por “*Texas Instruments*” fue en punto fijo. En la actualidad, un gran número de DSP’s están manejando aritmética en punto flotante.

Los procesadores digitales de Señales por lo general tienen una unidad llamada “*DSP Engine*” y esta a su vez contiene una operación llamada MAC (*Multiply Accumulator*), que como su nombre lo indica, puede realizar multiplicaciones acumulativas. Esto es:

$$Y(k) = \sum_{n=0}^N A_n U(k-n) \quad \text{..... 6}$$

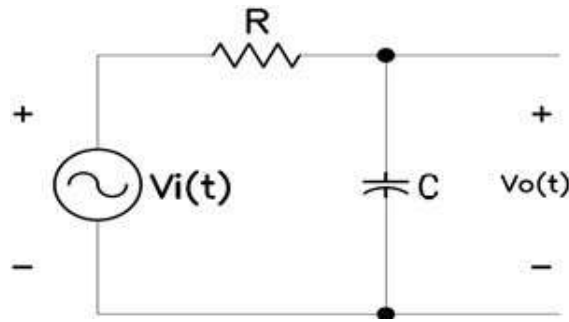
Donde $Y(k)$ es la salida, $U(k-n)$ es el conjunto de entradas y A_n es el set de coeficientes. Esta configuración es ideal para la implementación de filtros FIR (Finite Impulse Response).

Sin embargo, para poder implementar una función de transferencia $H(z)$ se requieren dos unidades MAC y por fortuna, todos los DSP contienen esas dos unidades por lo que:

$$Y(k) = \sum_{n=0}^N A_n U(k-n) + \sum_{m=1}^M B_m Y(k-m) \quad \text{..... 7}$$

Donde $Y(k)$ es la salida actual, $U(k-n)$ es el conjunto de entradas, A_n es el set de coeficientes de las entradas, B_m es el set de coeficientes asociados a las salidas e $Y(k-m)$ es el conjunto de salidas pasadas. Se puede concluir que la ecuación 7 representa a la ecuación en diferencias correspondiente a la ecuación 5.

Supongamos que estamos considerando un circuito RC de primer orden, cuya función de transferencia $H(s)$ es presentada en la ecuación 8:



$$H(s) = \frac{\frac{1}{RC}}{s + \frac{1}{RC}} = \frac{\frac{1}{\tau}}{s + \frac{1}{\tau}} = \frac{V_o(s)}{V_i(s)} = \frac{Y(s)}{U(s)} \quad \text{..... 8}$$

Donde τ representa a la constante de tiempo del sistema. Nótese que se está haciendo los cambios de variable de $Y(s) = V_o(s)$, $U(s) = V_i(s)$ para mantener la compatibilidad con las nomenclaturas de los textos teóricos.

Aunque se puede utilizar un comando directo para obtener la función de transferencia $H(z)$, se va a presentar todo el proceso utilizando el “Método de Tustin”, cuya relación es:

$$S = \left(\frac{2}{T}\right) \left(\frac{Z-1}{Z+1}\right) \quad \text{..... 9}$$

Donde “ T ” representa el tiempo de muestreo, el cual, deberá ser elegido adecuadamente para que su respuesta no se vea muy pixelada en su gráfica.

Sustituyendo la ecuación 9 en la ecuación 8, se tiene:

$$H(z) = \frac{\frac{1}{\tau}}{\left[\left(\frac{2}{T}\right)\left(\frac{Z-1}{Z+1}\right)\right] + \frac{1}{\tau}} \quad \text{..... 10}$$

Multiplicando por $T(Z+1)$ el numerador y denominador obtenemos:

$$H(z) = \frac{\frac{T(Z+1)}{\tau}}{2(Z-1) + \frac{T(Z+1)}{\tau}} \quad \text{..... 11}$$

Agrupando términos semejantes:

$$H(z) = \frac{\left(\frac{T}{\tau}\right)(Z+1)}{\left(2 + \frac{T}{\tau}\right)Z - \left(2 - \frac{T}{\tau}\right)} = \frac{\left(\frac{T}{\tau}\right)(Z+1)}{\left(\frac{2\tau+T}{\tau}\right)Z - \left(\frac{2\tau-T}{\tau}\right)} \quad \text{..... 12}$$

Dividiendo entre $\frac{\tau}{2\tau+T}$ se tiene:

$$H(z) = \frac{\left(\frac{T}{2\tau+T}\right)(Z+1)}{Z - \left(\frac{2\tau-T}{2\tau+T}\right)} \quad \text{..... 13}$$

Si consideramos que la frecuencia de muestreo es $f = 8\text{Khz}$, y la $\tau = 0.005\text{Seg}$, el tiempo de muestreo es $T = \frac{1}{f} = 0.000125\text{Seg}$, se obtendrá:

$$H(z) = \frac{\left(\frac{0.000125}{2(0.005)+0.000125}\right)(Z+1)}{Z - \left[\frac{2(0.005)-0.000125}{2(0.005)+0.000125}\right]} = \frac{0.01235(Z+1)}{Z-0.9753} \quad \text{..... 14}$$

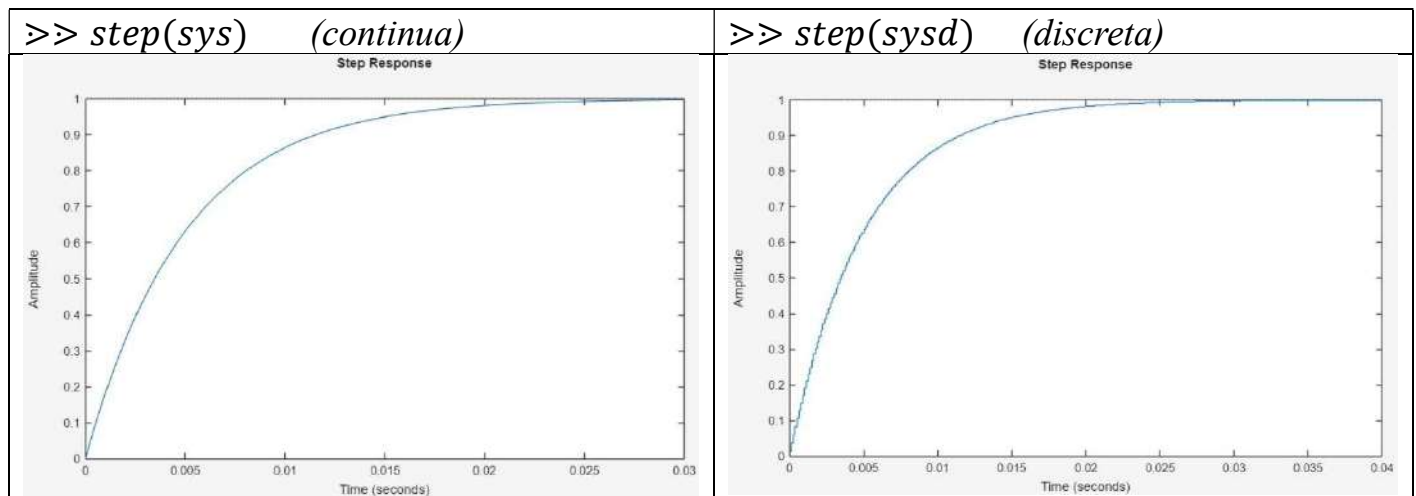
Ahora bien, si realizamos el mismo proceso utilizando MATLAB se tendrían los siguientes pasos:

`>> sys = tf([0 200],[1 200])` (que sería la función de transferencia continua)
`>> sysd = c2d(sys,0.000125,'tustin')` (de esta manera se obtendrá la $H(z)$ discreta)

Esto es:

$$H(z) = \frac{0.01235Z+0.01235}{Z-0.9753} \quad \text{..... 15}$$

Si comparamos las respuestas a escalón continua y discreta:



Nótese que con el tiempo de muestreo T elegido con respecto a la τ prácticamente no hay diferencia en las respuestas.

Para empatar la ecuación 15 con el formato de datos del simulador:

$$H(z) = \frac{0.01235Z + 0.01235}{Z - 0.9753} = \frac{A_0Z + A_1}{Z + B_1} \quad \text{..... 16}$$

Descomponiendo la ecuación 15 para obtener la ecuación en diferencias:

$$H(z) = \frac{0.01235}{1 - 0.9753Z^{-1}} = \frac{Y(z)}{U(z)} \quad \text{..... 17}$$

Descomponiendo en términos de entradas y salidas:

$$Y(z)[1 - 0.9753Z^{-1}] = U(z)[0.01235 + 0.01235Z^{-1}] \quad \text{..... 18}$$

Antitransformando se tiene:

$$Y(k) - 0.9753Y(k-1) = 0.01235U(k) + 0.01235U(k-1) \quad \text{..... 19}$$

Finalmente, la salida actual es:

$$Y(k) = 0.01235U(k) + 0.01235U(k-1) + 0.9753Y(k-1) \quad \text{..... 20}$$

Si hacemos compatible la ecuación 20 con el formato de datos de los campos del Simulador de Funciones de Transferencia Discretas:

$$Y(k) = A_0U(k) + A_1U(k-1) + A_2U(k-2) + B_1Y(k-2) + B_2U(k-2) \quad \text{..... 21}$$

Donde la asignación de queda:

$$A_0 = 0.01235$$

$$A_1 = 0.01235$$

$$A_2 = 0.0$$

$$B_1 = 0.9753$$

$$B_2 = 0.0$$

Note que, para mantener la compatibilidad con la ecuación de segundo orden del simulador, se están asignando los coeficientes A_2 y B_1 se les está asignando valor igual a cero.

En la interfaz del simulador es posible asignar la frecuencia de entrada que se aplicará como señal de entrada $U(k)$, la cual, será una señal cuadrada y esta será generado por el módulo Raspberry Pi Pico 2 (RP2350) utilizando su segundo núcleo. El primer núcleo será el responsable de dar solución a la ecuación en diferencias. En términos generales, el software de la RP2350 estará constituyendo un proceso paralelo.



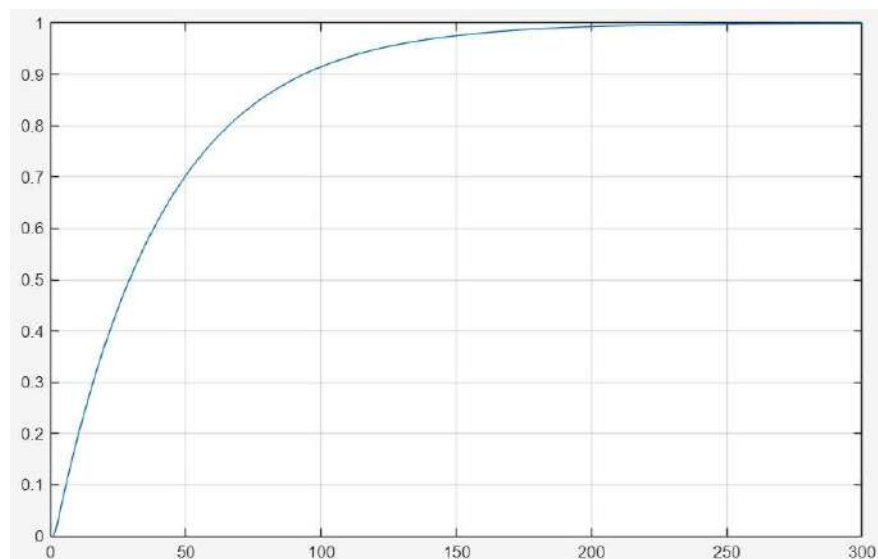
El funcionamiento de la interfaz requiere tener conectado y declarado el nombre del módulo bluetooth pulsando el botón “CONECTAR”, asignar frecuencia de entrada, llenar los campos de coeficientes correspondientes y luego pulsar el botón “GRAFICAR”. El proceso de graficado solo dura 30 segundos, lo cual, permitirá visualizar el comportamiento de la función de transferencia. Después de ese lapso podrá modificar los coeficientes si lo desea. Las graficas se presentan en dominio del tiempo y frecuencia.

Simulación mediante un Script de MATLAB:

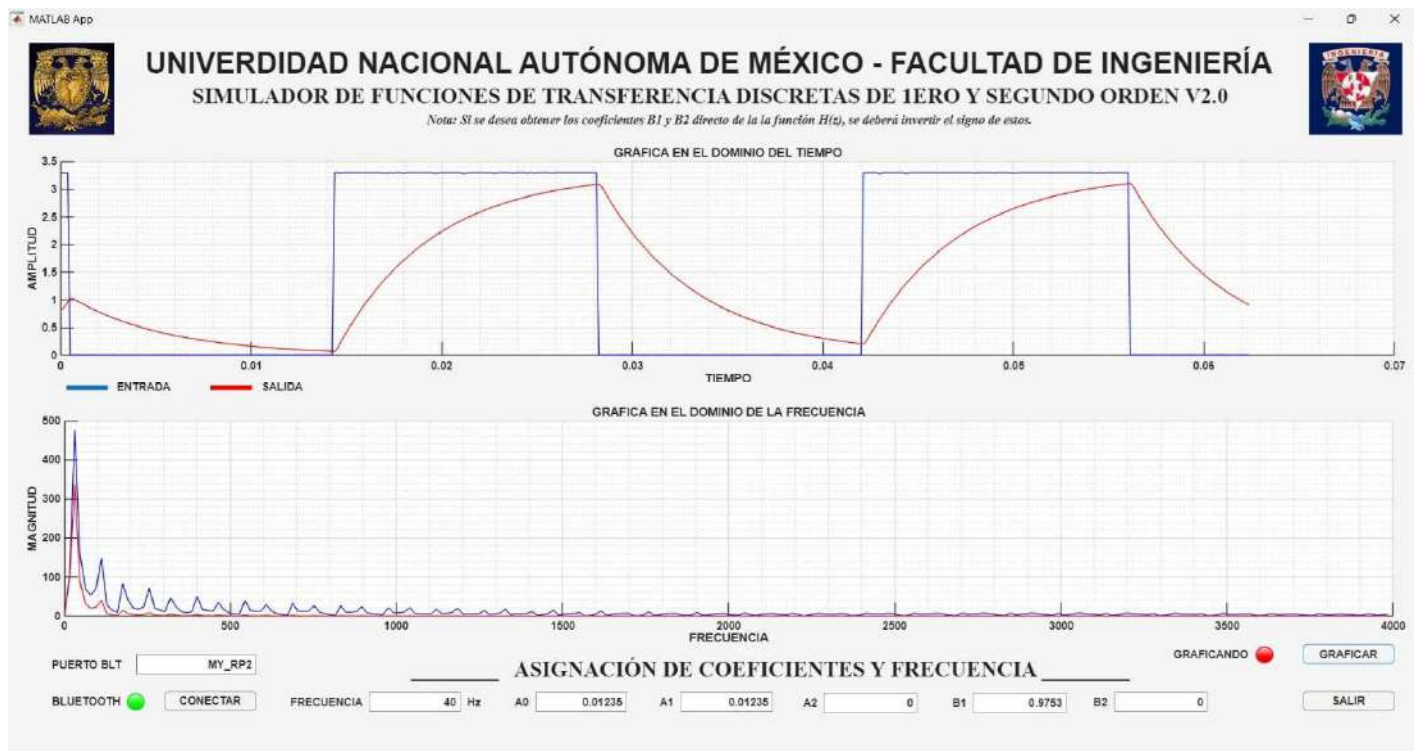
```

1 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
2 % SIMULACIÓN DE LA ECUACIÓN EN DIFERENCIAS UTILIZANDO UN SCRIPT DE MATLAB %
3 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4 %
5 %          A0*Z^2 + A1*Z + A2      Y(z)
6 %      H(z) = ----- = -----
7 %          Z^2 + B1*Z + B2      U(z)
8 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
9 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
10
11 % definición de entradas y salidas:
12 UK = 0;      % U(k) = 0
13 UK1 = 0;     % U(k-1) = 0
14 UK2 = 0;     % U(k-2) = 0
15 YK = 0;      % Y(k) = 0
16 YK1 = 0;     % Y(k-1) = 0
17 YK2 = 0;     % Y(k-2) = 0
18
19 % Definición de coeficientes:
20 A0 = 0.01235;
21 A1 = 0.01235;
22 A2 = 0.0;
23 B1 = 0.9753;
24 B2 = 0.0;
25
26 ESCALON = 1.0;
27
28 % xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx %
29 % Solución de la Ecuación en Diferencias "Genérica" para 1ero y 2do %
30 % orden. %
31 % xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx %
32 for i=1:300
33     YK = A0*UK + A1*UK1 + A2*UK2 + B1*YK1 + B2*YK2;
34     COLECT(i) = YK;      % Colecta las soluciones.
35
36     % Actualiza entradas / Salidas para próxima iteración:
37     UK2 = UK1;      % U(k-1) --> U(k-2)
38     UK1 = UK;       % U(k)  --> U(k-1)
39     YK2 = YK1;      % Y(k-1) --> Y(k-2)
40     YK1 = YK;       % Y(k)  --> Y(k-1)
41     UK = ESCALON;   % La nueva entrada.
42
43 end
44
45 plot(COLECT)
46 grid on

```



Respuesta en Tiempo Real para una Señal de Entrada de 40 Hz



7. Cuestionario Previo

- Explique cual es la diferencia entre un muestreador de señales simple y un muestreo con retén.
- De acuerdo con el Teorema del Muestreo nos dice que para recuperar la señal original, esta deberá ser muestreada al doble de la frecuencia. Observe las gráficas que ejemplifican el concepto y establezca sus conclusiones. Luego observe el esquema que muestra la relación de un canal de audio discreto y explique por qué en este caso se usa también el concepto del doble de la frecuencia con f_s (tiene que ver con la percepción del audio).

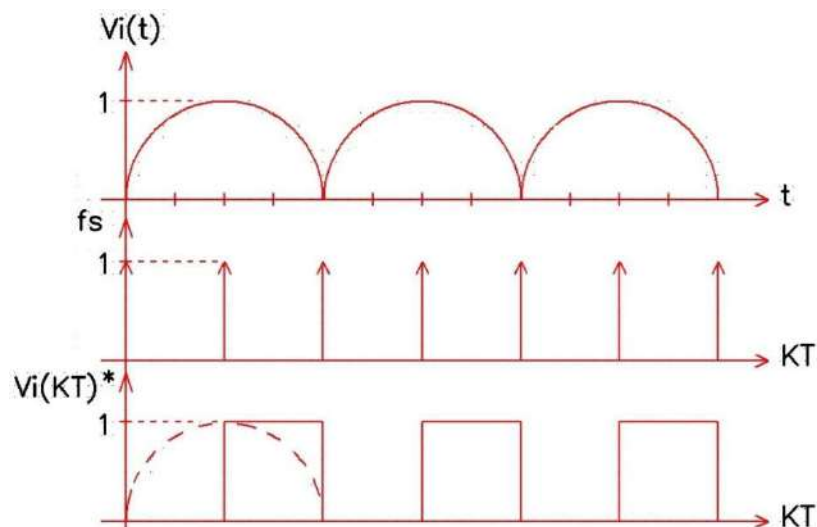


Figura 7.1 Acción del Muestreo con Retén



Figura 7.2 Respuesta Dinámica Teórica de un Canal de Audio

- c) Se sabe que la representación de una onda cuadrada mediante una secuencia de armónicos corresponde a una sumatoria infinita de armónicos (como se muestra en la ecuación 7.1). Si esta señal alimentara a un filtro pasa banda sintonizado en la frecuencia fundamental de la señal, ¿Cuál sería la respuesta de filtro a su salida y como sería la forma de onda resultante?

$$f(t) = \text{sen}(\omega_o t) + \left(\frac{1}{3}\right) \text{sen}(3\omega_o t) + \left(\frac{1}{5}\right) \text{sen}(5\omega_o t) + \left(\frac{1}{7}\right) \text{sen}(7\omega_o t) + \dots$$

- d) Acorde con la respuesta del inciso “a”, si un sistema discreto contara con una acción de muestreo pobre, ¿qué efecto tendría sobre la señal de salida?

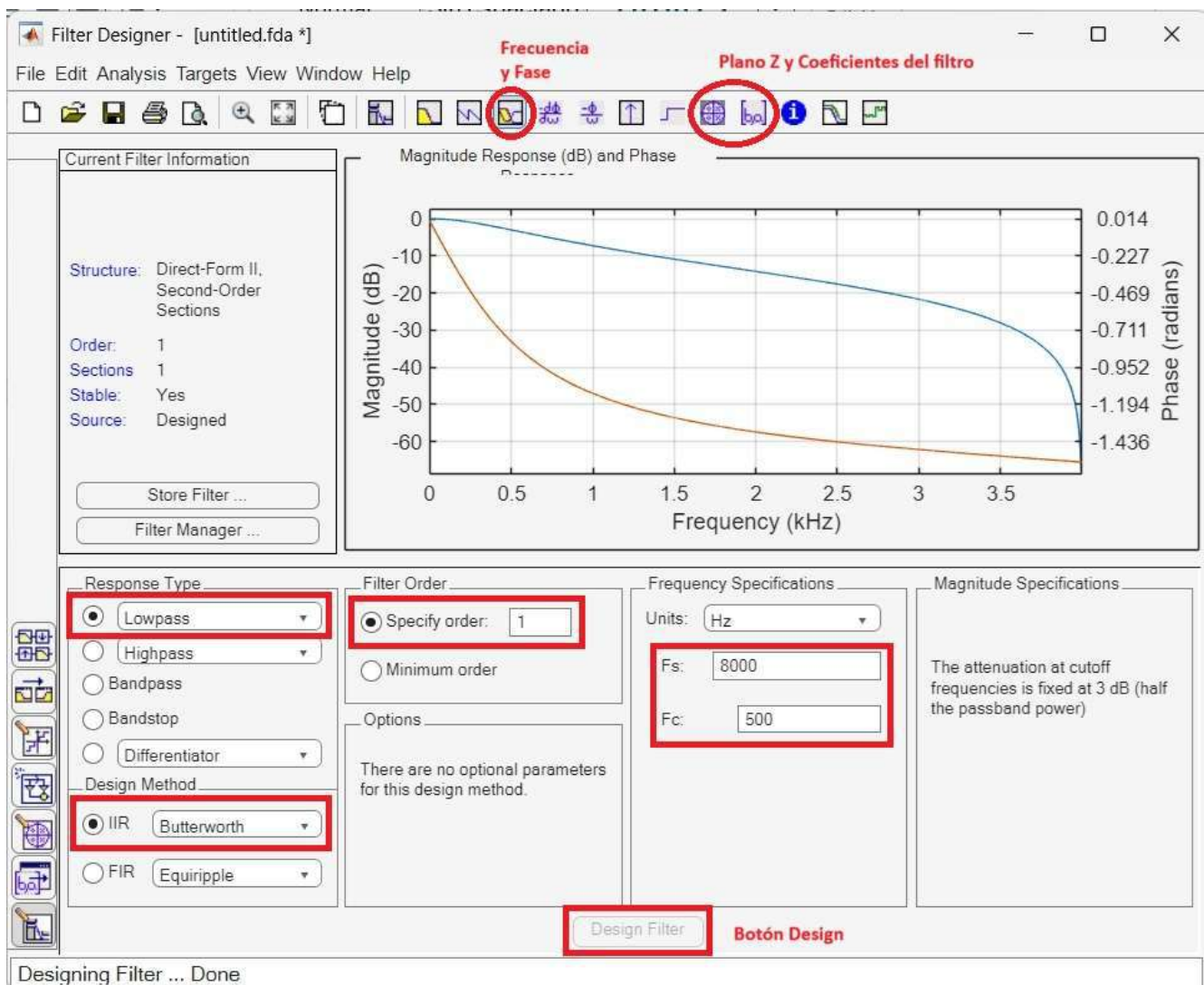
8. Actividades de la Práctica

La herramienta “*fdatool*” de MATLAB (“*filterDesigner*” en versiones posteriores), es una herramienta poderosa para generar filtros digitales bajo plantilla de diseño. Esto nos permite diseñar filtros FIR (Finite Impulse Response) y filtro IIR (Infinite Impulse Response). **Nota:** para inicial la aplicación deberá escribir el comando “*fdatool*”.

Para nuestros fines y dado que el simulador está pensado para trabajar con funciones de transferencia discretas, el diseño de filtros IIR nos va a ser de mucha utilidad.

8.1 Simulación de un Filtro Pasa Bajas de Primer Orden

A continuación, se diseñará un filtro IIR Butterworth Pasa Bajas de primer orden, cuya frecuencia de corte será igual a $f_c = 500\text{Hz}$.



Si pulsamos el botón de coeficientes observará que los datos tienen el siguiente formato:

```

-----
Section #1
-----
Numerator:
1
1
0
Denominator:
1
-0.668178637919298878955487452913075685501
0
Gain:
0.165910681040350532766680657914548646659
-----
Output Gain:
1

```

La función de transferencia se obtendrá de la siguiente manera:

$$H(z) = \left[\frac{\text{Numerator}}{\text{Denominator}} \right] * \text{Gain} * (\text{Output Gain})$$

$$H(z) = \left[\frac{Z + 1}{Z - 0.66817863791} \right] * (0.1659104036) * (1)$$

$$H(z) = \frac{0.1659104036Z + 0.1659104036}{Z - 0.66817863791}$$

Si se desea copiar directamente omitiendo el paso de la obtención de la ecuación en diferencias, los campos de coeficientes en el Simulador quedarían:

$$A_0 = 0.1659104036$$

$$A_1 = 0.1659104036$$

$$A_2 = 0.0$$

$$B_1 = (-0.66817863791) * (-1.0) = +0.66817863791$$

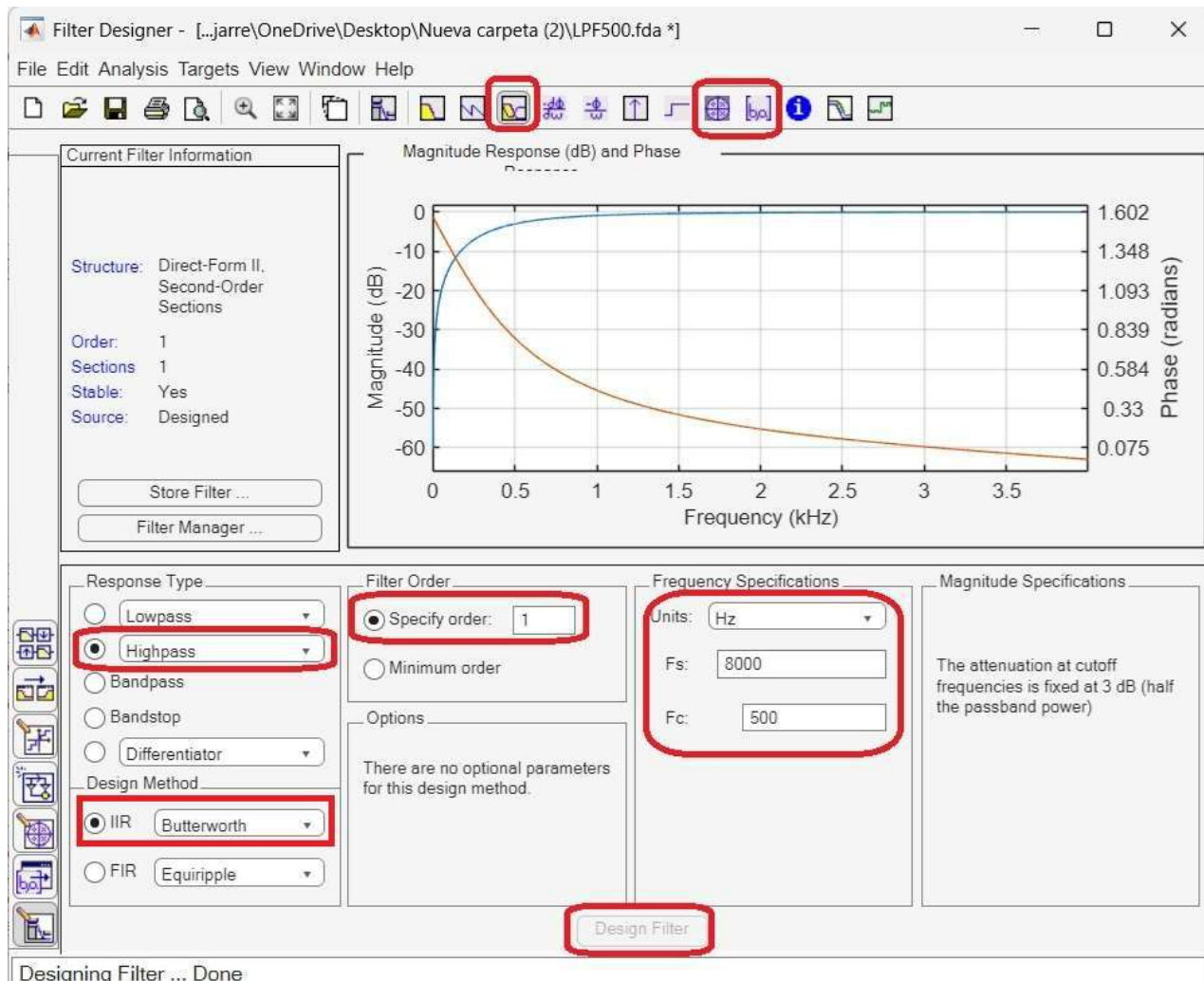
$$B_2 = (0.0) * (-1.0) = 0.0$$

Elija una frecuencia entre 20 a 100 Hz para la señal de entrada y pulse el botón “GRAFICAR” del simulador y observe resultados.

Finalmente pulse el botón que corresponde al “Plano Z” y observe el patrón de polos y cero del filtro.

8.2 Simulación de un Filtro Pasa Altas de Primer Orden

A continuación, se diseñará un filtro IIR Butterworth Pasa Altas de primer orden, cuya frecuencia de corte será igual a $f_c = 500\text{Hz}$.



Siguiendo los mismos pasos del ejemplo anterior, la función de transferencia resultante es:

$$H(z) = \frac{0.8340893189Z - 0.8340893189}{Z - 0.66817863791}$$

$$A_0 = 0.8340893189$$

$$A_1 = -0.8340893189$$

$$A_2 = 0.0$$

$$B_1 = (-0.66817863791) * (-1.0) = 0.66817863791$$

$$B_2 = (0.0) * (-1.0) = 0.0$$

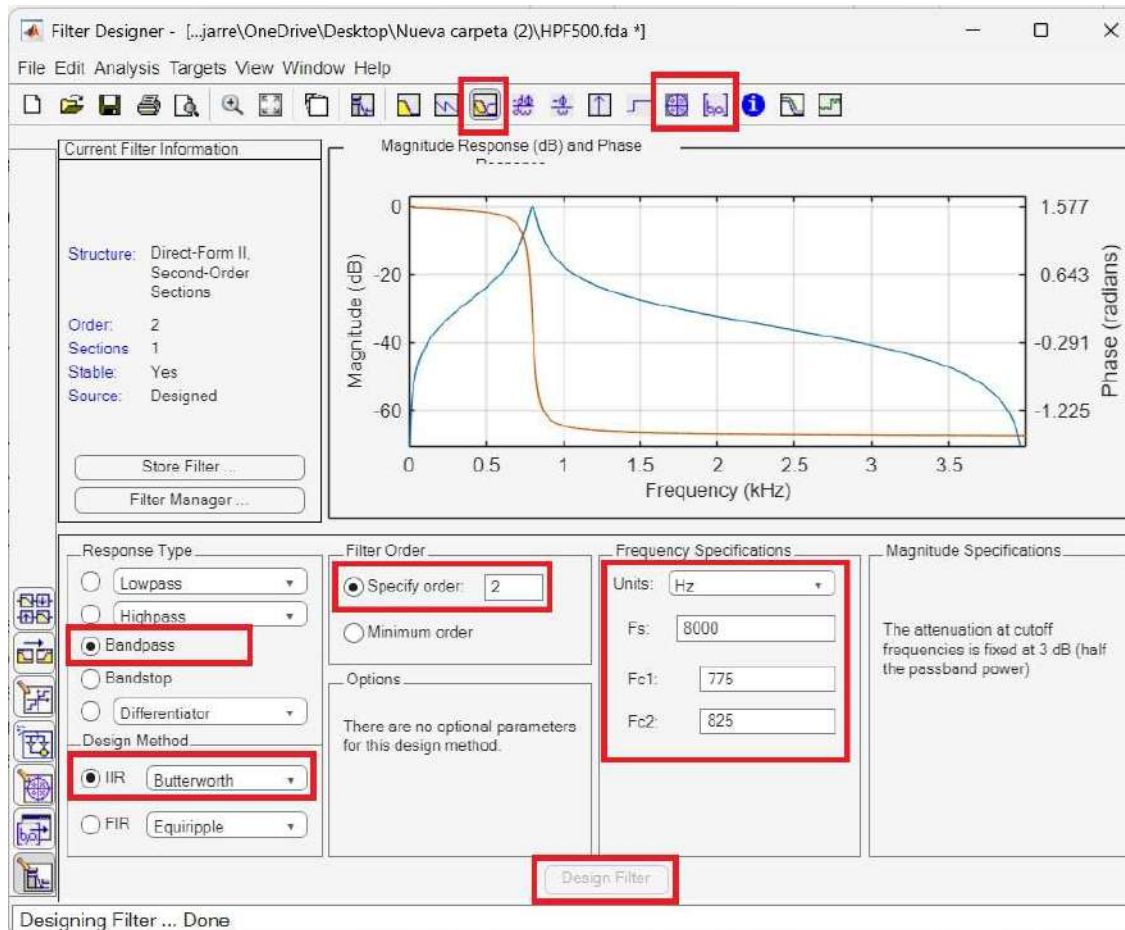
Elija una frecuencia entre 20 a 100 Hz para la señal de entrada y pulse el botón “GRAFICAR” del simulador y observe resultados.

Finalmente pulse el botón que corresponde al “Plano Z” y observe el patrón de polos y cero del filtro.

8.3 Simulación de un Filtro Pasa Banda de Segundo Orden

Ahora se va a diseñar un Filtro Pasa Banda Butterworth de Segundo Orden cuya frecuencia central de una $f_o = 600 \text{ Hz}$, con un ancho de banda $BW = 50 \text{ Hz}$.

De acuerdo al formato de la herramienta “fdatool” se tiene que especificar $f_{c1} = 575 \text{ Hz}$ y $f_{c2} = 625 \text{ Hz}$ para cumplir con los requisitos del ancho de banda.



Los coeficientes que presenta “fdatool” serían:

Section #1

Numerator:

1

0

-1

Denominator:

1

-1.748029719812035365222868676937650889158

0.961481451595328384662764165113912895322

Gain:

0.019259274202335863179769148700870573521

Output Gain:

1

De acuerdo a lo anterior se tiene:

$$H(z) = \left[\frac{Z^2 + 0 \cdot Z - 1}{Z^2 - 1.748029719812Z + 0.961481451595} \right] * (0.019259274202335) * (1.0)$$

$$H(z) = \frac{0.019259274202335Z^2 - 0.019259274202335}{Z^2 - 1.748029719812Z + 0.961481451595}$$

EL campo de coeficientes del simulador quedaría:

$$A_0 = 0.019259274202335$$

$$A_1 = 0.0$$

$$A_2 = -0.019259274202335$$

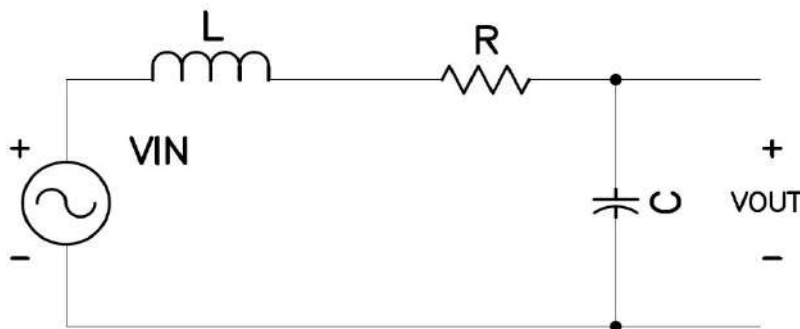
$$B_1 = (-1.748029719812) * (-1.0) = 1.748029719812$$

$$B_2 = (0.961481451595) * (-1.0) = -0.961481451595$$

- Elija una frecuencia de 800 Hz para la señal de entrada y pulse el botón “GRAFICAR” del simulador y observe resultados. Varie la frecuencia de entrada y observe los cambios que presenta la respuesta.
- Explique porque aparece solo una espiga en la respuesta en frecuencia de salida del filtro.
- Finalmente pulse el botón que corresponde al “Plano Z” y observe el patrón de polos y cero del filtro.

8.4 Simulación de un Sistema de Segundo Orden Subamortiguado

Considere el Modelo de un circuito LRC como el que muestra en la figura considerando que la frecuencia real de oscilación es $f = 500\text{Hz}$, con un sobrepaso $M_p = 70\%$



La función de transferencia $H(s)$ para este modelo sería:

$$H(s) = \frac{V_o(s)}{V_i(s)} = \frac{\frac{1}{LC}}{s^2 + \left(\frac{R}{L}\right)s + \left(\frac{1}{LC}\right)}$$

Como lo que deseamos es que esa función se comporte como un sistema subamortiguado, equivalentemente se podría decir que la función de transferencia anterior es equivalente a:

$$H(s) = \frac{\omega_o^2}{s^2 + 2\xi\omega_o s + \omega_o^2}$$

Dado que sólo tenemos como dato a la frecuencia real de oscilación y el sobrepaso máximo tendríamos que realizar varios pasos para poder conocer los valores asociados a la función de transferencia.

Como se tiene como dato el sobrepaso Mp y este está definido por la fórmula:

$$Mp = \left[\text{Exp} \left(\frac{-\xi\pi}{\sqrt{1-\xi^2}} \right) \right] * 100$$

Despejando el coeficiente de amortiguamiento ξ se tiene:

$$\text{Exp} \left(\frac{-\xi\pi}{\sqrt{1-\xi^2}} \right) = \frac{Mp}{100}$$

$$\frac{-\xi\pi}{\sqrt{1-\xi^2}} = \log \eta \left(\frac{Mp}{100} \right)$$

$$-\xi\pi = \left[\sqrt{1-\xi^2} \right] * \log \eta \left(\frac{Mp}{100} \right)$$

Como el sobrepaso deseado es del 70%, sustituyendo valores se tiene:

$$-\xi\pi = \left[\sqrt{1-\xi^2} \right] * \log \eta(0.7)$$

$$-\xi\pi = \sqrt{1-\xi^2} * (-0.3566749439)$$

Elevando al cuadrado se tiene:

$$\xi^2 \pi^2 = (1-\xi^2) * (0.1272170156)$$

$$\xi^2 \pi^2 + 0.1272170156 \xi^2 = 0.1272170156$$

$$\xi^2 (\pi^2 + 0.1272170156) = 0.1272170156$$

$$\xi^2 = \frac{0.1272170156}{\pi^2 + 0.1272170156}$$

Finalmente, el coeficiente de amortiguamiento resultante es:

$$\xi = \sqrt{\frac{0.1272170156}{\pi^2 + 0.1272170156}} = 0.1128084507$$

Proponiendo una frecuencia real de oscilación $f = 500$ Hertz, de acuerdo con la ecuación se tiene entonces:

$$\omega = 2 * \pi * f = 3141.592 \left[\frac{rad}{seg} \right]$$

La relación de ks frecuencia natural de oscilación esta dada como:

$$\omega = \omega_o \sqrt{1 - \xi^2}$$

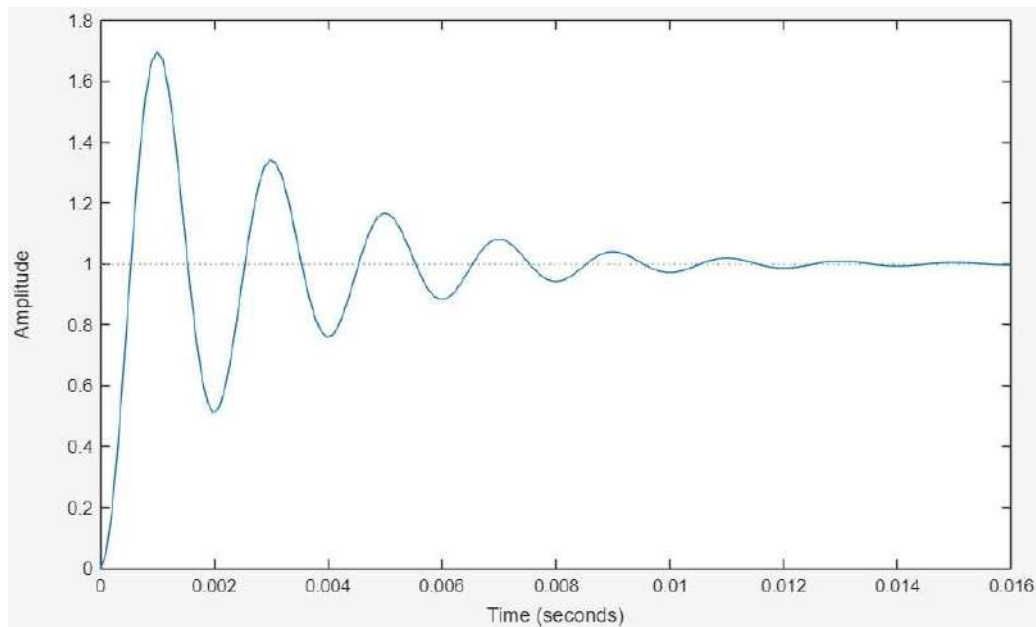
$$\omega_o = \frac{\omega}{\sqrt{1 - \xi^2}} = \frac{3141.592}{\sqrt{1 - (0.1128084507)^2}} = \frac{3141.592}{0.99361675} = 3161.774 \left[\frac{rad}{seg} \right]$$

Por lo tanto, llenando los campos requeridos en la función de transferencia continua son:

$$H(s) = \frac{9996817.2571}{s^2 + 713.349s + 9996817.2571}$$

La respuesta escalón utilizando MATLAB:

```
>> sys = tf([0 0 9996817.2571],[1 713.349 9996817.2571]);
>> step(sys)
```



Llevando al ámbito discreto el sistema continuo con MATLAB se tiene:

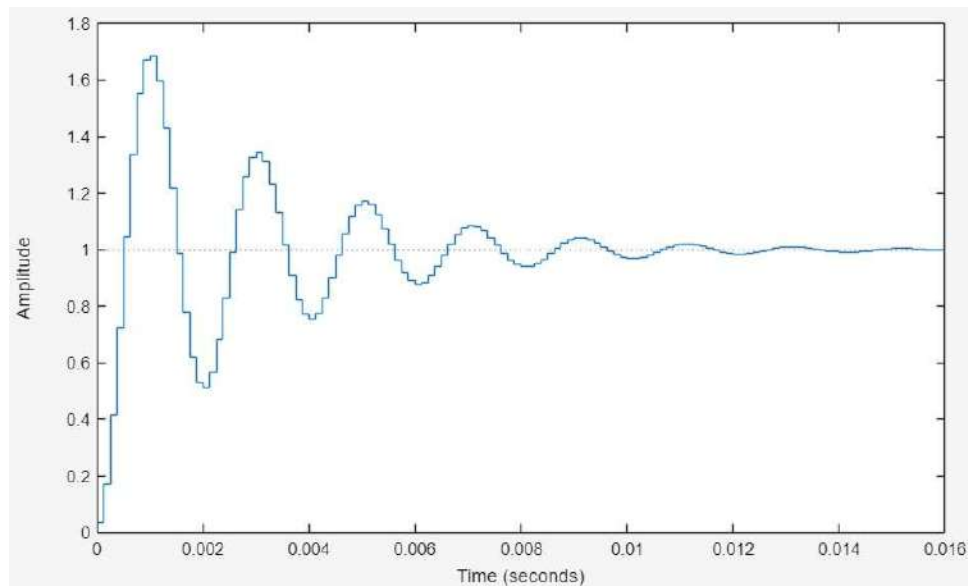
`>> sysd = c2d(sys, 0.000125, 'tustin')` *(sin punto y coma al final)*

Donde $T = \frac{1}{f_s} = \frac{1}{8000} = 125\mu\text{Seg}$

$$H(z) = \frac{0.03604Z^2 + 0.07207Z + 0.03604}{Z^2 - 1.774Z + 0.9177}$$

Obteniendo su respuesta a escalón discreto:

`>> step(sysd)`



NOTA IMPORTANTE: Inherentemente, la Raspberry Pi Pico 2 (RP2350) puede presentar una acción de filtro pasa bajas, por lo que, las discontinuidades que se ven en la gráfica estarán redondeadas.

Tomando como base a la función de transferencia $H(z)$, el campo de coeficientes del simulador quedaría como:

$$A_0 = 0.03604$$

$$A_1 = 0.1707$$

$$A_2 = 0.03604$$

$$B_1 = (-1.774) * (-1.0) = 1.7740$$

$$B_2 = (0.9177) * (-1.0) = -0.9177$$

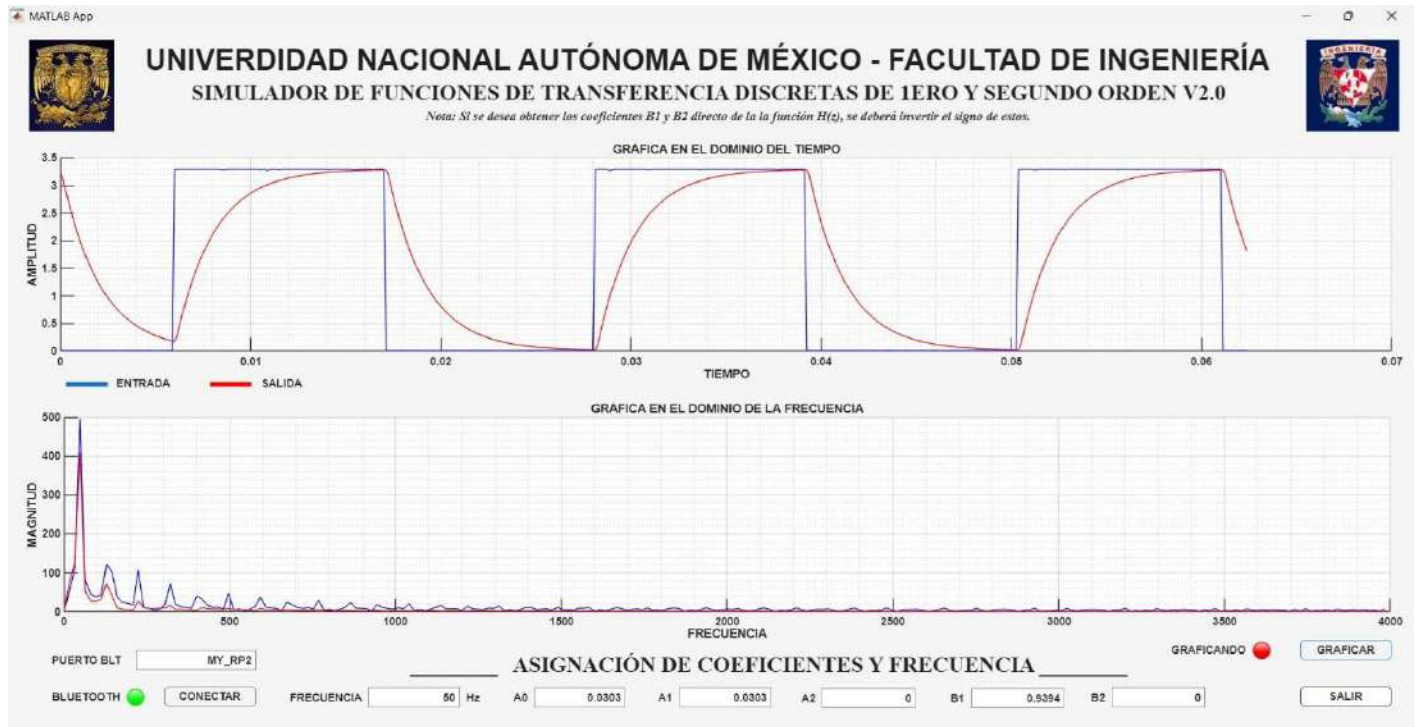
- a) Elija una frecuencia de entrada entre 20 a 100 Hz en donde pueda verse correctamente el subamortiguamiento del sistema.
- b) Escriba un script en MATLAB que simule la respuesta a escalón para la ecuación en diferencias resultante de la función de transferencia.
- c) Finalmente pulse el botón que corresponde al “Plano Z” y observe el patrón de polos y cero de la función de transferencia $H(z)$.

NOTA IMPORTANTE: pueden haber notado que en los ejemplos anteriores se eligieron constantes numéricas con un número importante de decimales. Esto se debe a que se están trabajando con aproximaciones numéricas y por la misma razón, si se recortan decimales se corre el riesgo de incrementar el error numérico.

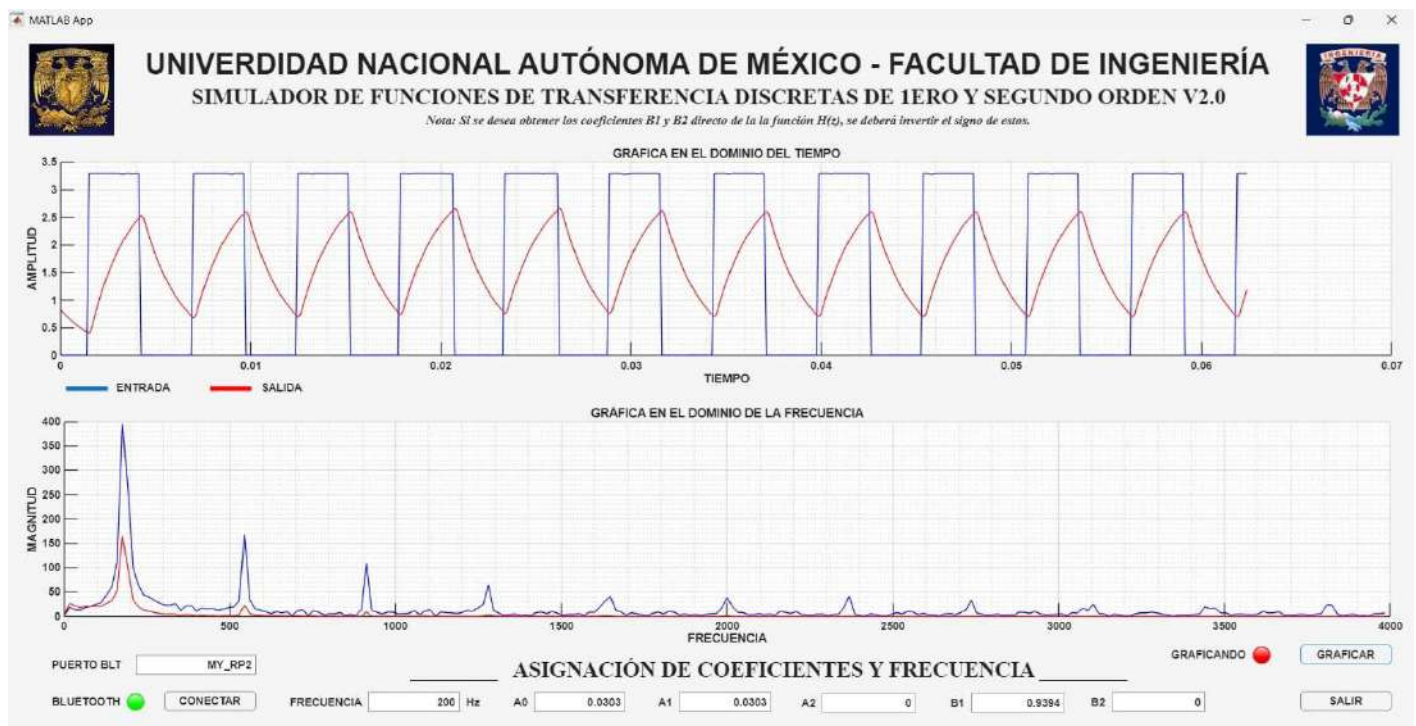
9. Solución de las Actividades de la Práctica

9.1 Solución de la Actividad 8.1

Copie y pegue los coeficientes obtenidos en la actividad 8.1, fije la frecuencia de entradas a 50 Hz. Deberá ver la siguiente gráfica:

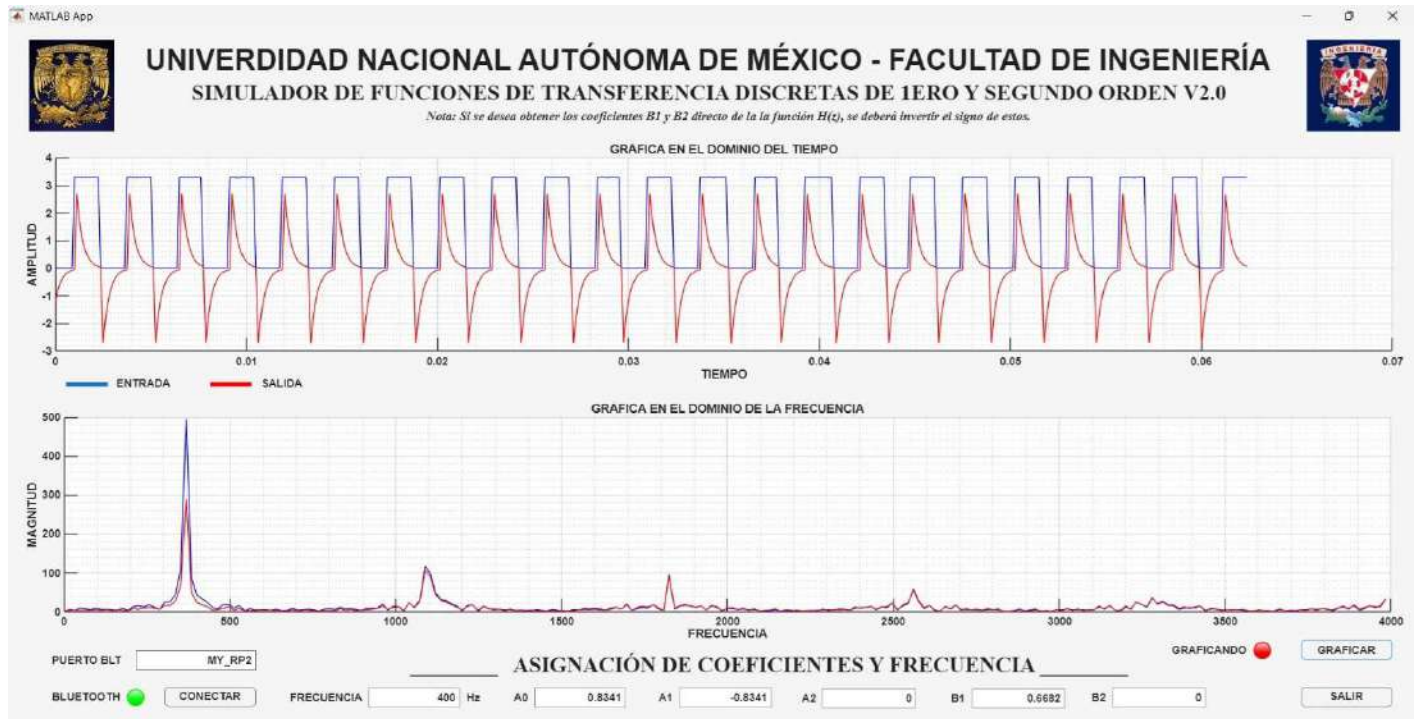


¿Dadas las características pasa – bajas que pasaría si aplico una señal de entrada de 200 Hz? Observe la gráfica y justifique sus conclusiones.

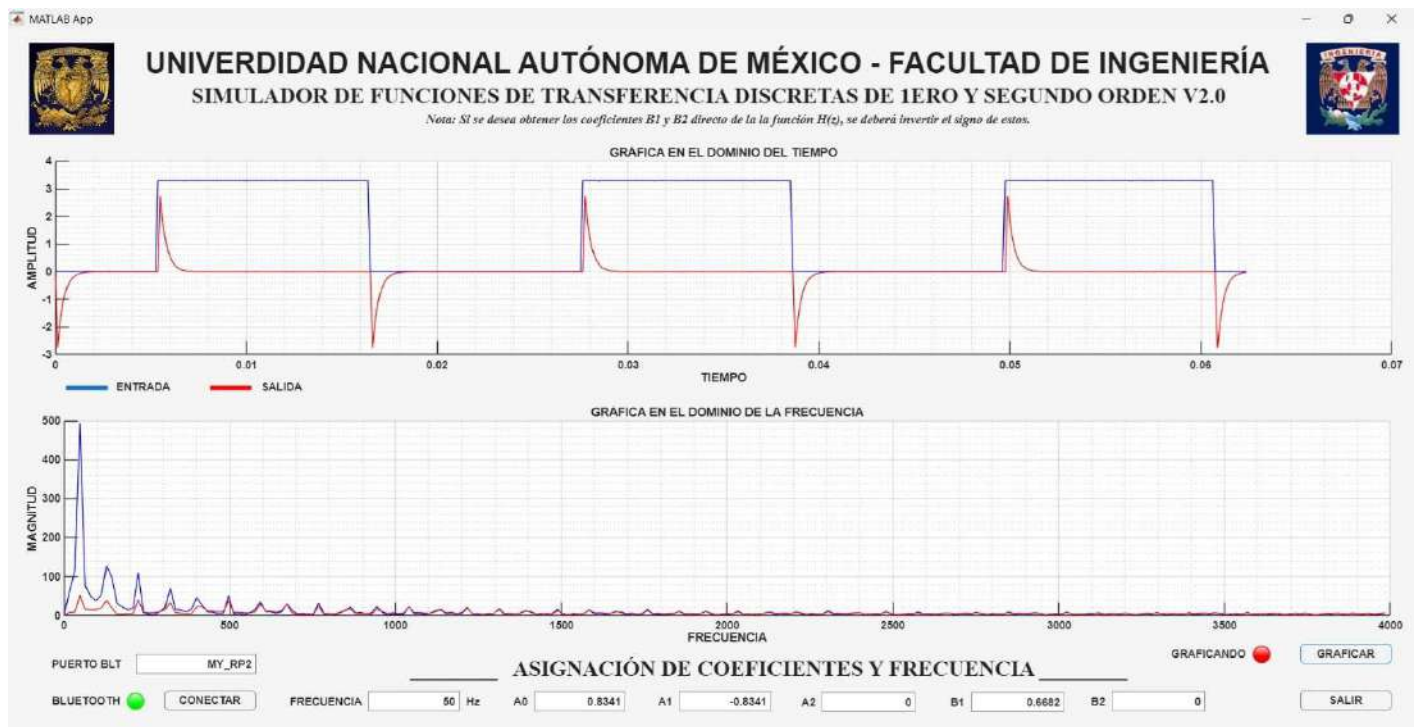


9.2 Solución de la Actividad 8.2

Copie y pegue los coeficientes obtenidos en la actividad 8.1, fije la frecuencia de entradas a 400 Hz. Deberá ver la siguiente gráfica:

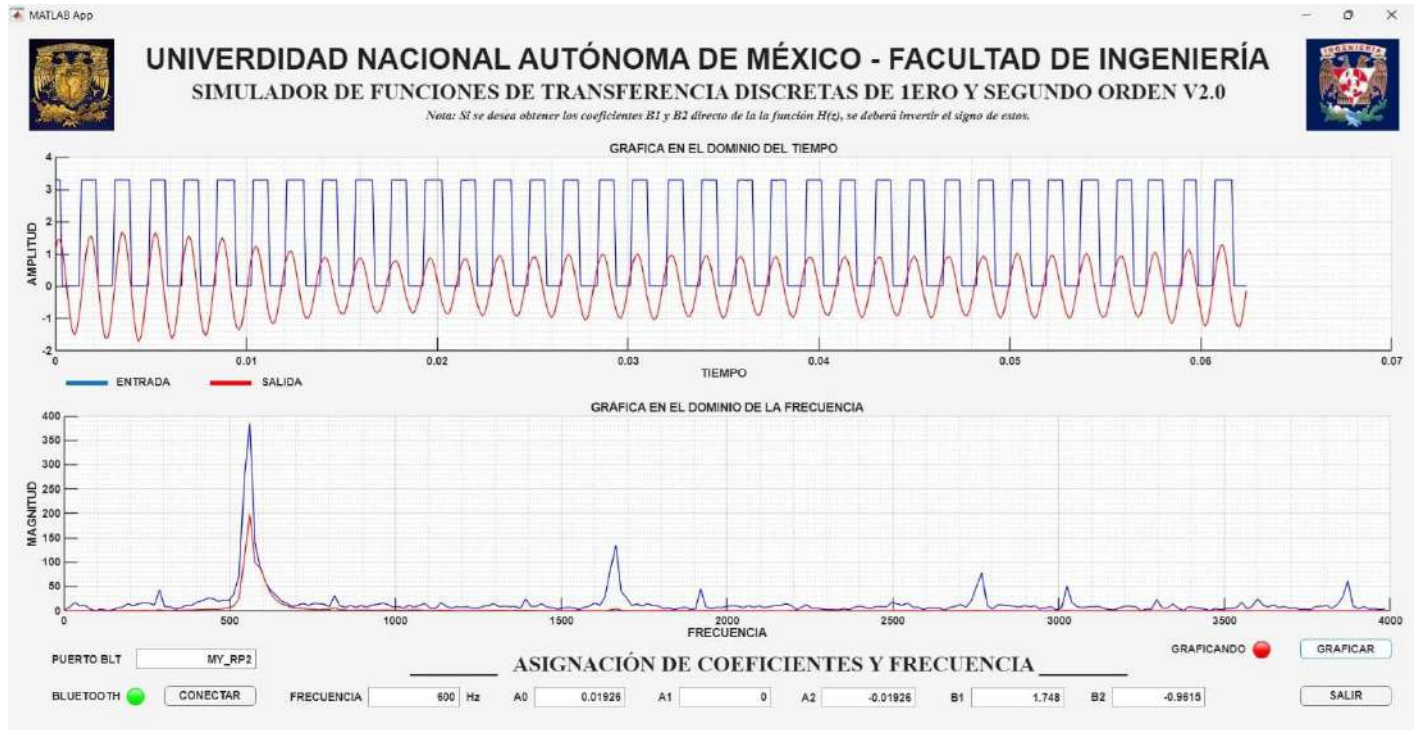


¿Dadas las características pasa – altas que pasaría si aplico una señal de entrada de 50 Hz? Observe la gráfica y justifique sus conclusiones.

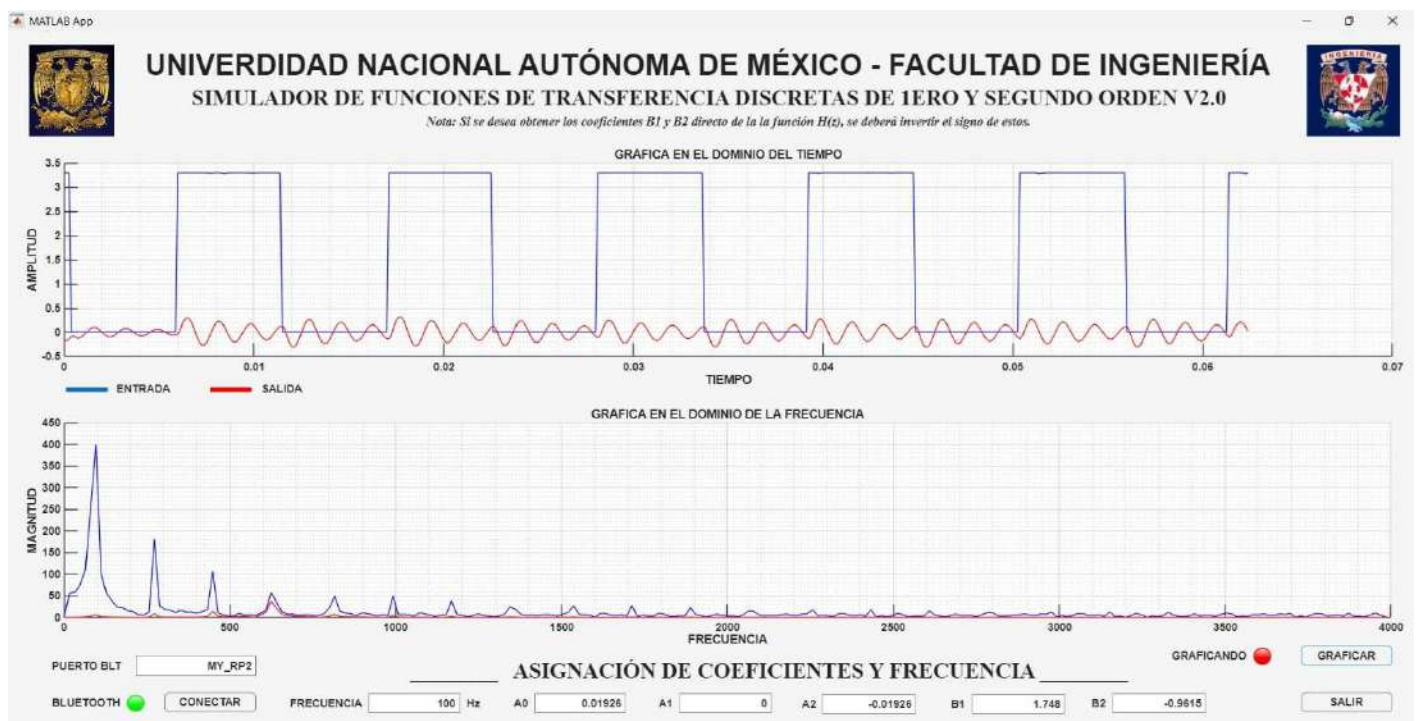


9.3 Solución de la Actividad 8.3

Copie los coeficientes sugeridos en 8.3 y aplique una señal de entrada que coincida con la frecuencia $f_0 = 600\text{Hz}$ del filtro pasa – banda y observe la señal de salida. Emita sus conclusiones.

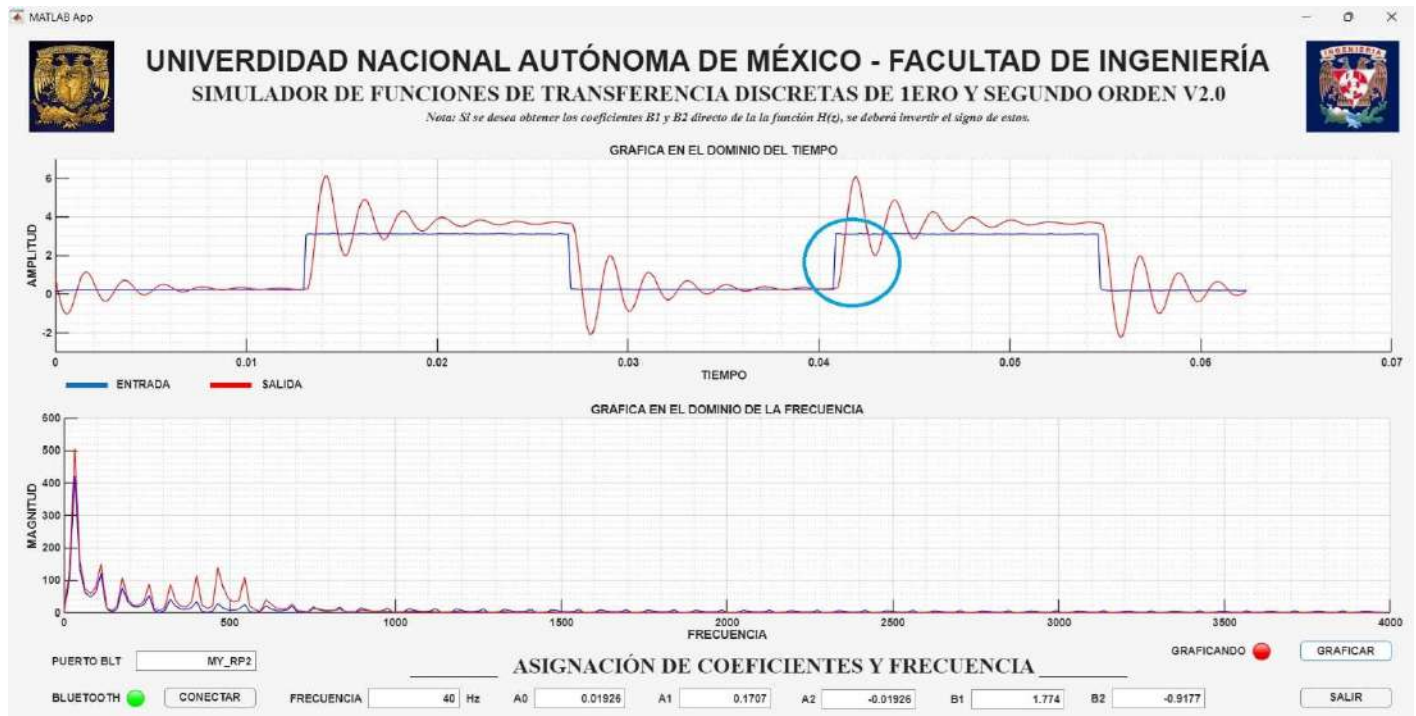


Cambie la frecuencia de entrada a 200 Hz y emita sus conclusiones al respecto.



9.4 Solución de la actividad 8.4

Copie y pegue los coeficientes obtenidos en la actividad 8.4, fije la frecuencia de entradas a 50 Hz. Deberá ver la siguiente gráfica:



Aunque es prácticamente imperceptible, puede observarse un pequeño retardo entre el escalón de subida de la señal de entrada y la señal de salida. Esto se debe al “*tiempo máquina*” que ocupa la Raspberry Pi Pico 2 (RP2350) para dar solución al algoritmo de la ecuación en diferencias. En procesadores tales como el dsPIC33FJ128 que tienen un reloj de 40 Mhz este retardo es muy notable. En el caso de la RP2350 su reloj está en un “*overclock*” de 270 Mhz, por lo que, este retardo es mínimo.

NOTA IMPORTANTE: algunas ocasiones la APP de MATLAB puede aparecer un mensaje de error “*dato no numérico*” al pegar los coeficientes. Una solución inmediata podría ser copiar el coeficiente en la línea de comando de MATLAB + ENTER y luego copiarlo en el campo correspondiente de la APP.

